
KIWI

Proyecto Informe Parcial



Integrantes
Aarón Timaná Castro
Jean Piero Huaylla Huillca
Mateo Espinoza Monti

Link:

https://github.com/JeanPiero369/08C_DevOps_07S03_Proyecto.git

Curso: IS4024 – DevOps

Profesor: Wilson Julca Mejia

20 de septiembre de 2026

Índice

1	Título y descripción del proyecto	1
1.1	Nombre del producto	1
1.2	El problema y la solución propuesta	1
1.3	Integrantes y roles	1
2	Justificación de la elección del tema	2
2.1	Contexto del proyecto	2
2.2	Motivación y relevancia	2
3	Arquitectura del sistema	3
4	Control de versiones con Git y Conventional Commits	4
4.1	Qué son los Conventional Commits y por qué se usan	4
4.2	Tipos principales	4
4.3	Metodología de ramificación	4
4.4	Historial de commits	4
4.5	Análisis detallado de una muestra de commits	6
4.6	Pull Requests	7
5	Pruebas unitarias	9
5.1	Implementación actual	9
5.2	Evidencia de ejecución	9
6	Automatización con Bash	10
7	Dockerización de la aplicación	11
7.1	Dockerfiles	11
7.2	Docker Compose	11
7.3	Evidencia de ejecución	11
8	Conclusiones	13

1 Título y descripción del proyecto

1.1 Nombre del producto

El proyecto se llama **KIWI**, una plataforma de refuerzo educativo para el curso de Comunicación de nivel secundaria, pensada para funcionar tanto como aplicación web como a través de un asistente conversacional por WhatsApp. El nombre resume el propósito del producto: una fruta pequeña, cotidiana y nutritiva, en la misma línea que la app busca ser un refuerzo breve y constante del aprendizaje, no una plataforma pesada que compita con el tiempo de clase.

1.2 El problema y la solución propuesta

El diagnóstico que origina el proyecto viene de una entrevista con la docente Sara, encargada del curso de Comunicación en la Institución Educativa Fe y Alegría. Los estudiantes de primer año de secundaria presentan un desempeño bajo en expresión oral, vocabulario, inferencia lectora y redacción de oraciones coherentes. A esa dificultad académica se suma una limitación de acceso: buena parte del alumnado no cuenta con una computadora o tableta propia, y quienes tienen acceso a un celular con frecuencia lo comparten con su familia. Cualquier solución tenía entonces que ser liviana, funcionar en celulares de gama baja y no depender de que el estudiante tenga un dispositivo exclusivo para usarla.

KIWI responde a esto con dos frentes. Para el estudiante, una aplicación con quizzes generados automáticamente por inteligencia artificial a partir de texto o de archivos que sube el propio alumno o el docente, retroalimentación inmediata al terminar cada quiz, un sistema de rachas y ranking que gamifica la práctica diaria, un registro emocional breve al iniciar el día y un asistente de IA accesible por WhatsApp para quienes no puedan instalar o abrir la aplicación web. Para el docente, un panel que centraliza el desempeño de sus alumnos, el ranking del aula, la creación de desafíos diarios y la revisión de los quizzes que la IA generó antes de asignarlos, de modo que la automatización acelera su trabajo sin quitarle la posibilidad de intervenir.

Técnicamente, la solución está construida como un conjunto de microservicios independientes, cada uno responsable de un dominio del negocio, que se comunican a través de un orquestador y se exponen al usuario final mediante una aplicación web. La Sección 3 describe esa arquitectura con más detalle.

1.3 Integrantes y roles

Los tres roles son responsabilidades principales y no compartimentos estancos: el trabajo de integración entre servicios, la revisión de Pull Requests y la definición de requisitos se hizo de forma conjunta entre los tres integrantes.

Tabla 1: Integrantes del equipo y rol principal dentro del proyecto.

Integrante	Rol
Aarón Timaná Castro	Backend Lead. Diseño y desarrollo de la lógica de negocio de los microservicios, modelado de datos y definición de contratos de API entre servicios.
Jean Piero Huaylla Huilca	DevOps Lead. Control de versiones, flujo de ramas y Pull Requests, contenerización de los servicios y automatización del entorno de desarrollo e integración.
Mateo Espinoza Monti	Frontend Lead. Diseño de la experiencia de usuario, desarrollo de la aplicación web y consumo de la API expuesta por el orquestador.

2 Justificación de la elección del tema

2.1 Contexto del proyecto



Figura 1: Estudiantes de la Institución Educativa Fe y Alegría, contexto que motivó el proyecto KIWI.

El contexto educativo peruano en el que se ubica tiene una particularidad que condiciona cada decisión técnica: los estudiantes de la institución tienen acceso tecnológico limitado y comparten dispositivos, así que la solución no puede asumir una computadora personal ni una conexión estable, y tiene que llegar también por un canal que ya usan a diario, como WhatsApp.

2.2 Motivación y relevancia

El tema se eligió por dos razones que se refuerzan entre sí. La primera es de impacto social directo: comunicación oral y escrita es una competencia transversal, y reforzarla en la transición de primaria a secundaria tiene efecto sobre el resto de cursos, no solo sobre uno. La segunda es de relevancia técnica para el curso de DevOps: un sistema con cinco componentes que deben desplegarse juntos (frontend, tres microservicios y un orquestador)

es exactamente el escenario en el que rendir cuentas de un buen control de versiones, pruebas automatizadas y contenerización deja de ser un ejercicio de forma y se vuelve una necesidad operativa. Cada microservicio tiene su propio lenguaje y runtime (Java, Node.js, Python), lo que además obliga a que la estrategia de Docker y de pruebas se adapte a cada stack en lugar de aplicar una receta única, un problema representativo de lo que ocurre en proyectos reales con equipos y tecnologías heterogéneas.

3 Arquitectura del sistema

El sistema sigue una arquitectura de microservicios con un orquestador que actúa como puerta de entrada única para el frontend. La Figura 2 resume los componentes y su tecnología. El frontend nunca llama directamente a un microservicio de dominio: todas las peticiones pasan por **ms-orchestrator**, que compone las respuestas de los servicios internos y expone una sola API hacia afuera.

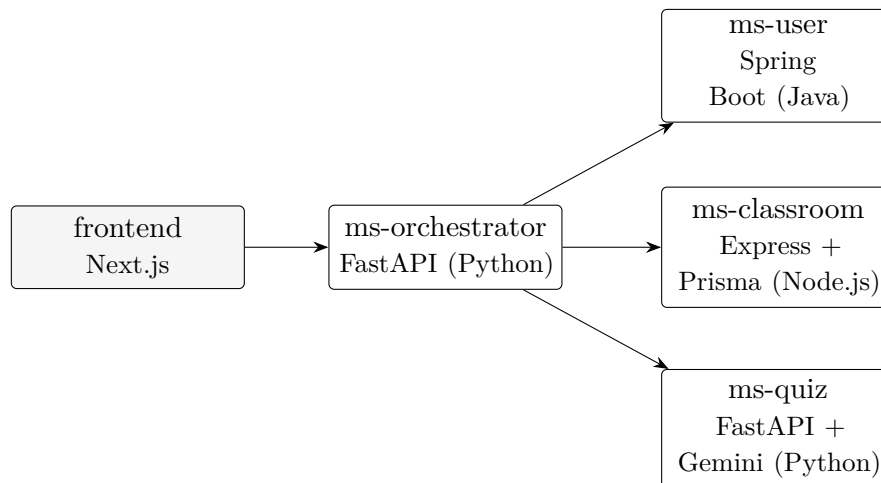


Figura 2: Arquitectura de KIWI. El frontend consume exclusivamente al orquestador, que a su vez distribuye cada petición al microservicio responsable del dominio.

Componente	Tecnología	Responsabilidad
frontend	Next.js / TypeScript	Interfaz web para docentes y estudiantes.
ms-orchestrator	FastAPI / Python	API Gateway; enruta y compone llamadas.
ms-user	Spring Boot / Java	Autenticación y gestión de usuarios.
ms-classroom	Express + Prisma / Node.js	Aulas, estudiantes y competencias.
ms-quiz	FastAPI + Gemini / Python	Generación y gestión de quizzes.
PostgreSQL	PostgreSQL 13	Persistencia compartida por esquemas/lógica de dominio.

Tabla 2: Componentes principales de KIWI.

4 Control de versiones con Git y Conventional Commits

4.1 Qué son los Conventional Commits y por qué se usan

Conventional Commits es una convención para escribir mensajes de commit con una estructura fija: `tipo(alcance): descripción`, donde el tipo clasifica la naturaleza del cambio y el alcance, opcional, indica qué parte del sistema afecta. El equipo la adoptó por tres razones concretas. Primero, con cinco componentes independientes (frontend y cuatro servicios de backend), el alcance del commit dice de inmediato en qué parte del sistema hay que mirar sin necesidad de abrir el diff. Segundo, separa con claridad el trabajo de construir una funcionalidad nueva (**feat**) del de corregir un defecto (**fix**) o del de tareas de infraestructura (**chore**), lo que vuelve legible el historial incluso para alguien que no participó en el desarrollo. Tercero, es el estándar sobre el que se apoyan herramientas de generación automática de *changelogs* y de versionado semántico, algo relevante para un sistema que va a seguir integrando componentes en la segunda mitad del curso.

4.2 Tipos principales

4.3 Metodología de ramificación

El equipo sigue **Trunk Based Development (TBD)**: existe una única rama tronco, **main**, que se mantiene siempre en un estado integrable, y cada microservicio o característica se desarrolla en una rama de corta duración creada directamente desde **main** (**feat/microservice-user**, **feat/microservice-classroom**, **feat/microservice-quiz**, **feat/orchestrator**, **feat/frontend**, **chore/docker-compose**). Esa rama temporal vive solo mientras dura la funcionalidad, se integra a **main** mediante un Pull Request y se elimina inmediatamente después del merge; no hay ramas de larga duración como **develop** o **release**, ni una jerarquía de ramas entre sí, a diferencia de GitFlow. Esto es justamente lo que muestra el historial de la Sección 4.4: cada Pull Request nace de **main** y vuelve a **main**, nunca de otra rama de feature.

4.4 Historial de commits

El repositorio cuenta con 26 commits en **main** al momento de este informe. El Código 1 reproduce el historial completo obtenido con `git log -oneline`.

```
1 825474f Merge pull request #6 from JeanPiero369/chore/docker-compose
2 3a3222b fix: stabilize service containers and return created quizzes
3 2047719 Merge pull request #5 from JeanPiero369/feat/frontend
4 9efe0b5 feat(frontend): add web application and core features
5 08466b1 Merge pull request #4 from JeanPiero369/feat/orchestrator
6 9b4fffe chore(ms-orchestrator): add container setup
7 7327962 feat(ms-orchestrator): add orchestration service
8 cdcbabd Merge pull request #3 from JeanPiero369/feat/microservice-quiz
9 1f0275b feat(ms-quiz): add quiz management features
10 5307ba5 Merge pull request #2 from JeanPiero369/feat/microservice-classroom
11 8115984 feat(ms-classroom): add classroom and student management
12 5d9fdc9 Merge pull request #1 from JeanPiero369/feat/microservice-user
13 f279c23 feat(ms-user): add authentication and user management
```

Tabla 3: Tipos de Conventional Commits empleados en el historial del proyecto.

Tipo	Uso y ejemplo del historial
feat	Funcionalidad nueva y completa, casi siempre un microservicio entero al incorporarse por primera vez. <i>Ejemplo:</i> <code>feat(ms-quiz): add quiz management features</code>
fix	Corrección de un defecto sobre código ya existente, sin agregar funcionalidad nueva. <i>Ejemplo:</i> <code>fix: stabilize service containers and return created quizzes</code>
docs	Cambios que afectan únicamente a la documentación (README, guías, comentarios de API), sin tocar el código de ejecución. <i>Ejemplo:</i> <code>docs(readme): describe local setup with docker compose</code>
test	Incorporación o modificación de pruebas unitarias, de integración o de extremo a extremo, sin alterar el comportamiento del sistema. <i>Ejemplo:</i> <code>test(ms-quiz): add unit tests for quiz service</code>
chore	Tareas de mantenimiento o infraestructura que no cambian el comportamiento de negocio (setup de contenedores, configuración, dependencias). <i>Ejemplo:</i> <code>chore(ms-orchestrator): add container setup</code>
ci	Cambios en la configuración o los scripts de integración y entrega continua (pipelines, workflows, automatización de despliegue). <i>Ejemplo:</i> <code>ci: add GitHub Actions workflow to build and test services</code>
refactor	Reestructuración interna del código para mejorar su legibilidad o mantenibilidad, sin corregir defectos ni añadir funcionalidad. <i>Ejemplo:</i> <code>refactor(ms-quiz): extract validation logic into separate module</code>

14 eflaeca Initial commit

Código 1: Historial de commits en main (`git log -oneline`).

Commits

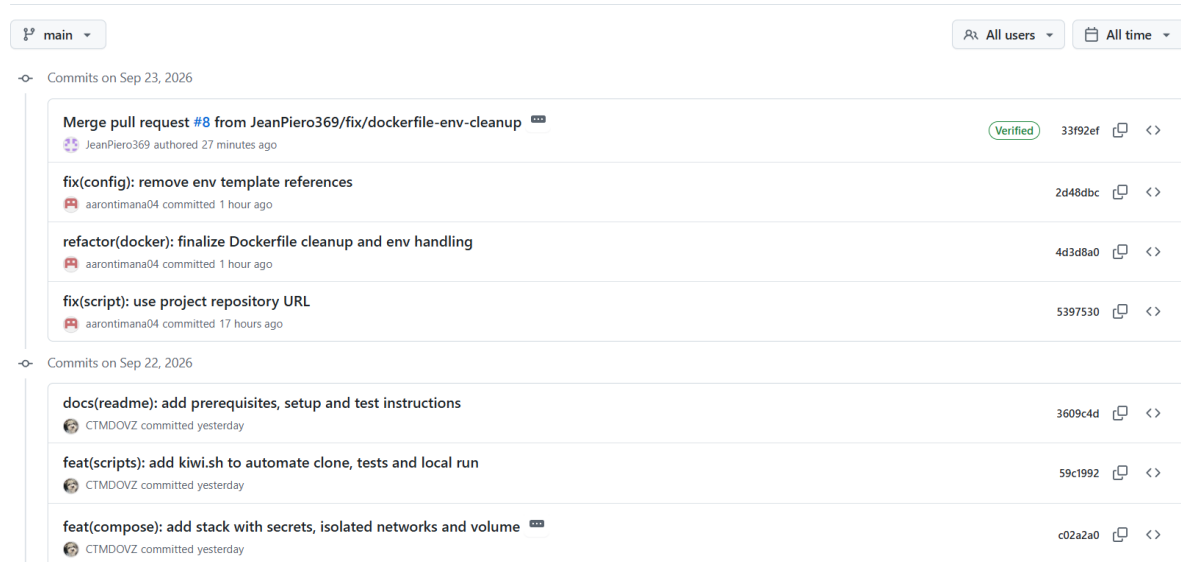


Figura 3: Historial de Commits.

4.5 Análisis detallado de una muestra de commits

De los 26 commits del historial se seleccionaron 6 para un análisis más detallado, priorizando los que corresponden a la incorporación de cada microservicio, que son los que más información aportan sobre cómo creció el sistema.

f279c23 — feat(ms-user): add authentication and user management

Primer microservicio incorporado al repositorio: 63 archivos y 2,125 líneas añadidas. Crea el proyecto Spring Boot completo (`pom.xml`, estructura Maven), su `Dockerfile` multi-etapa y la colección de peticiones de prueba manual (`api-endpoints/`) organizada en `Auth`, `Student` y `Teacher`. Establece el patrón que seguirán los demás microservicios: cada uno se agrega en un commit `feat` único que deja el servicio funcional y contenerizable desde el primer momento.

8115984 — feat(ms-classroom): add classroom and student management

Segundo microservicio, en Node.js con Express y Prisma: 43 archivos y 2,756 líneas. Introduce el esquema de base de datos (`prisma/schema.prisma`), las rutas de creación y consulta de aulas y la asociación de estudiantes, docentes y competencias a una clase. Es el servicio que mantiene la relación entre usuarios y quizzes asignados, y por eso es el que más endpoints de asociación expone (agregar estudiante, agregar docente, asociar competencia, asociar quiz).

1f0275b — feat(ms-quiz): add quiz management features

Es el commit individual más grande del historial: 32 archivos y 4,901 líneas. Implementa la generación de quizzes con el modelo Gemini 2.0 Flash a partir de texto o de un PDF

cargado por el usuario, la gestión de preguntas y respuestas, y el registro de intentos de un estudiante. Su tamaño refleja que es el microservicio con más lógica de negocio propia (procesamiento de PDF, llamadas al modelo de IA, cálculo de puntaje), a diferencia de los otros dos, que son mayormente CRUD.

7327962 — feat(ms-orchestrator): add orchestration service

Agrega la puerta de entrada en FastAPI: 51 archivos y 2,183 líneas, con un controlador por dominio (`auth_controller`, `student_controller`, `teacher_controller`, `classroom_controller`, `quiz_controller`) que reenvía las peticiones del frontend a `ms-user`, `ms-classroom` y `ms-quiz` usando `httpx`. Este commit es el punto en el que el sistema deja de ser tres servicios sueltos y pasa a tener una única API coherente hacia afuera.

9efe0b5 — feat(frontend): add web application and core features

El commit más grande de todo el historial: 95 archivos y 22,844 líneas (la mayor parte, `package-lock.json`). Agrega la aplicación Next.js completa: el panel del docente (`dashboard/classrooms`, `dashboard/competencias`), el layout general y el cliente HTTP que consume al orquestador. Con este commit el sistema queda usable de punta a punta por primera vez, ya que hasta entonces los cuatro servicios de backend solo se podían probar con las colecciones de Bruno.

3a3222b — fix: stabilize service containers and return created quizzes

Único commit de tipo `fix` del historial, con 7 archivos y 29 inserciones frente a 18 eliminaciones. Corrige dos problemas distintos. El primero es de contenerización: los `Dockerfile` de `ms-quiz`, `ms-classroom`, `ms-orchestrator` y `ms-user` copiaban varios directorios de código fuente en una sola instrucción `COPY`, y se separó cada uno en su propia instrucción para evitar errores de build; de paso se reemplazó la imagen base `python:3.11-slim-buster`, ya sin soporte, por `python:3.11-slim`. El segundo es un defecto funcional real: el endpoint `createQuiz` de `ms-orchestrator` armaba la respuesta del microservicio de quizzes pero nunca la retornaba (`response.json()` se llamaba sin `return`), así que el frontend recibía un cuerpo vacío tras crear un quiz; y en el cliente HTTP del frontend, la llamada `createQuiz` envolvía la petición en un `try/catch` que absorbía cualquier error en silencio. Este commit es el ejemplo más claro del historial de por qué separar `fix` de `feat` importa: es un cambio pequeño y localizado, fácil de revisar en un Pull Request, que no habría sido visible si estuviera mezclado dentro de un commit de funcionalidad.

4.6 Pull Requests

Cada rama de funcionalidad se integró a `main` mediante un Pull Request, siguiendo la metodología de ramificación descrita antes. La Tabla 4 resume los seis Pull Requests del repositorio.

Tabla 4: Pull Requests del repositorio y rama de origen.

PR	Rama	Contenido
#1	feat/microservice-user	Autenticación y gestión de usuarios (ms-user)
#2	feat/microservice-classroom	Gestión de aulas y estudiantes (ms-classroom)
#3	feat/microservice-quiz	Generación y gestión de quizzes (ms-quiz)
#4	feat/orchestrator	Servicio orquestador (ms-orchestrator)
#5	feat/frontend	Aplicación web (frontend)
#6	chore/docker-compose	Estabilización de contenedores de cada servicio

All pull requests

New pull request

is:pr state:closed

☐ Open	0	☒ Closed	8	Author	Label	Projects	Milestones	Reviews	Assignee	Sort	Newest	Menu
☐		Fix/dockerfile env cleanup	#8 · aarontimana04 opened 1 hour ago · Closed 25 minutes ago · Collaborator · Approved									
☐		test, refactor, feat: add unit tests, harden Dockerfiles and compose	#7 · CTMDOVZ opened yesterday · Closed 27 minutes ago · Collaborator · Review required									
☐		fix: stabilize service containers and return created quizzes	#6 · JeanPiero369 opened yesterday · Closed yesterday · Owner · Approved									
☐		feat(frontend): add web application and core features	#5 · JeanPiero369 opened 2 days ago · Closed 2 days ago · Owner · Approved									
☐		feat/orchestrator	#4 · JeanPiero369 opened 2 days ago · Closed 2 days ago · Owner · Approved									
☐		feat(ms-quiz): add quiz management features	#3 · JeanPiero369 opened 2 days ago · Closed 2 days ago · Owner · Approved									
☐		feat(ms-classroom): add classroom and student management	#2 · JeanPiero369 opened 2 days ago · Closed 2 days ago · Owner · Approved									
☐		feat(ms-user): add authentication and user management										

Figura 4: Historial de pull request.

5 Pruebas unitarias

5.1 Implementación actual

La suite unitaria se concentra en `ms-orchestrator/tests/` y utiliza **pytest**. Para aislar el servicio de usuarios se usa **respx**, que intercepta las llamadas de `httpx`; por ello las pruebas no requieren levantar `ms-user` ni PostgreSQL.

Se implementaron **12 pruebas unitarias**: 7 validan DTOs de Pydantic y 5 ejercitan el controlador de autenticación.

- Validación de login correcto, email inválido y límites de longitud del password.
- Conversión y rechazo de roles inválidos.
- Valores por defecto del modelo de estudiante.
- Login exitoso con respuesta simulada del servicio de usuarios.
- Propagación de error 401.
- Respuesta 503 cuando el servicio de usuarios no está disponible.
- Validación 422 que evita llamar al servicio externo con un body inválido.
- Registro exitoso con respuesta simulada.

El comando documentado para ejecutarlas es:

```
1 ./kiwi.sh test
```

También pueden ejecutarse manualmente:

```
1 cd ms-orchestrator
2 python3 -m venv .venv
3 source .venv/bin/activate
4 pip install -r requirements-dev.txt
5 pytest -v
```

5.2 Evidencia de ejecución

La ejecución local recolectó 12 casos y finalizó con 12 **passed**. La advertencia visible corresponde a una deprecación de `starlette.testclient` y no representa una falla de las pruebas.

```

08466b1 Merge pull request #4 from JeanPiero369/feat/orchestrator
* 80466b1 (origin/feat/orchestrator) chore(ms-orchestrator): add container setup
* 7327962 feat(ms-orchestrator): add orchestration service
cdcbabd Merge pull request #3 from JeanPiero369/feat/microservice-quiz
* 1f0275b (origin/feat/microservice-quiz) feat(ms-quiz): add quiz management features
53b7ba5 Merge pull request #2 from JeanPiero369/feat/microservice-classroom
* 8115984 (origin/feat/microservice-classroom) feat(ms-classroom): add classroom and student management
5d9fcd9 Merge pull request #1 from JeanPiero369/feat/microservice-user
* f279c23 (origin/feat/microservice-user) feat(ms-user): add authentication and user management
* eflaeca Initial commit
usuario@MacBookPro: ~ % cd /Users/usuario/Documents/08C_DevOps_07503_Proyecto && git branch
* feature/tests-and-docker-hardening
main
usuario@MacBookPro: ~ % cd /Users/usuario/Documents/08C_DevOps_07503_Proyecto && ./kiwi.sh test
>> Ejecutando pruebas unitarias (ms-orchestrator)

test session starts
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/usuario/Documents/08C_DevOps_07503_Proyecto/ms-orchestrator/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/usuario/Documents/08C_DevOps_07503_Proyecto/ms-orchestrator
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1, respx-0.23.1
collected 12 items

tests/test_auth_controller.py::test_login_exitoso_devuelve_token PASSED [ 8%]
tests/test_auth_controller.py::test_login_propaga_401_del_servicio_de_usuarios PASSED [ 16%]
tests/test_auth_controller.py::test_login_devuelve_502_si_usuario_no_responde PASSED [ 25%]
tests/test_auth_controller.py::test_login_con_body_invalido_no_llama_al_servicio PASSED [ 33%]
tests/test_auth_controller.py::test_registro_exitoso_devuelve_token PASSED [ 41%]
tests/test_schemas.py::test_login_valido acepta email y password PASSED [ 50%]
tests/test_schemas.py::test_login rechaza email invalido PASSED [ 58%]
tests/test_schemas.py::test_login rechaza password menor a 3 caracteres PASSED [ 66%]
tests/test_schemas.py::test_login rechaza password mayor a 16 caracteres PASSED [ 75%]
tests/test_schemas.py::test_registro convierte role a enum PASSED [ 83%]
tests/test_schemas.py::test_registro rechaza role inexistente PASSED [ 91%]
tests/test_schemas.py::test_student_inicia_con_monedas_en_cero PASSED [100%]

===== warnings summary =====
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
/Users/usuario/Documents/08C_DevOps_07503_Proyecto/ms-orchestrator/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
12 passed, 1 warning in 0.05s

=====
NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
08C_devops_07503_proyecto-classrooms-1  microservice-classroom  "docker-entrypoint.s..."  classrooms  14 hours ago  Up 14 hours  3000/tcp
08C_devops_07503_proyecto-frontend-1    frontend              "docker-entrypoint.s..."  frontend    14 hours ago  Up 14 hours  0.0.0.0:5174->5174/tcp
08C_devops_07503_proyecto-orchestrator-1 orchestrator          "uvicorn main:app --..."  orchestrator 14 hours ago  Up 14 hours (healthy)  0.0.0.0:8080->8080/tcp
08C_devops_07503_proyecto-quizes-1      microservice-quiz      "uvicorn app:app --h..."  quizes      14 hours ago  Up 14 hours (healthy)  8081/tcp
08C_devops_07503_proyecto-users-1       microservice-users      "java -XX:MaxRAMPerm..."  users       14 hours ago  Up 14 hours  8080/tcp
shared_postgres_db_kiwi                 postgres:13-alpine     "docker-entrypoint.s..."  db           14 hours ago  Up 14 hours (healthy)

```

Figura 5: Ejecución de las 12 pruebas unitarias y estado de los contenedores mediante Docker Compose.

6 Automatización con Bash

El repositorio incorpora `kiwi.sh` en la raíz. El script unifica tareas repetitivas del entorno de desarrollo:

Comando	Acción
<code>./kiwi.sh clone</code>	Clona o actualiza el repositorio.
<code>./kiwi.sh setup</code>	Verifica que exista <code>.env</code> y genera <code>secrets/db_password.txt</code> a partir de <code>DB_PASSWORD</code> .
<code>./kiwi.sh test</code>	Crea/reutiliza un entorno virtual e inicia pytest.
<code>./kiwi.sh up</code>	Ejecuta la preparación y construye/levanta el stack con Docker Compose.
<code>./kiwi.sh down</code>	Detiene el stack conservando el volumen.
<code>./kiwi.sh logs</code>	Muestra logs del stack o de un servicio.
<code>./kiwi.sh status</code>	Muestra el estado de los contenedores.

Tabla 5: Automatización disponible en `kiwi.sh`.

7 Dockerización de la aplicación

7.1 Dockerfiles

Los componentes cuentan con Dockerfile adaptado a su runtime. La iteración final utiliza usuarios no root cuando corresponde, ordena las capas para favorecer el uso de caché y aplica builds multi-etapa en los servicios donde aporta una reducción del runtime. Durante la validación se mantuvieron instrucciones `COPY` separadas cuando eran necesarias para conservar la estructura de paquetes esperada por Python.

Servicio	Base principal	Puerto
frontend	Node.js / Next.js multi-etapa	3000
ms-user	Maven builder + Eclipse Temurin JRE	8080
ms-classroom	node:18-alpine	3000
ms-quiz	python:3.11-slim	8001
ms-orchestrator	python:3.11-slim	8000

Tabla 6: Resumen de contenerización.

7.2 Docker Compose

El archivo `compose.yml` define el stack completo. PostgreSQL utiliza un volumen nombrado para persistencia y un healthcheck con `pg_isready`. Los servicios de dominio esperan a que la base de datos esté saludable mediante `depends_on` con `condition: service_healthy`. La topología de redes es deliberada:

- **web:** conecta frontend con el orquestador.
- **backend:** conecta el orquestador con los microservicios internos.
- **data:** conecta microservicios con PostgreSQL y se declara `internal: true`.

Solo frontend y orquestador publican puertos al host. PostgreSQL no utiliza `ports`. La contraseña de la base se monta mediante un secret y el volumen `shared_db_data` conserva la información al ejecutar `docker compose down` sin `-v`.

7.3 Evidencia de ejecución

La evidencia de terminal de la Figura 5 muestra el stack levantado con `docker compose ps`; en esa validación los servicios se encontraban activos y los componentes con healthcheck reportaban estado saludable. Adicionalmente, se validó la exposición del orquestador mediante Swagger y el acceso al frontend de KIWI desde el navegador.

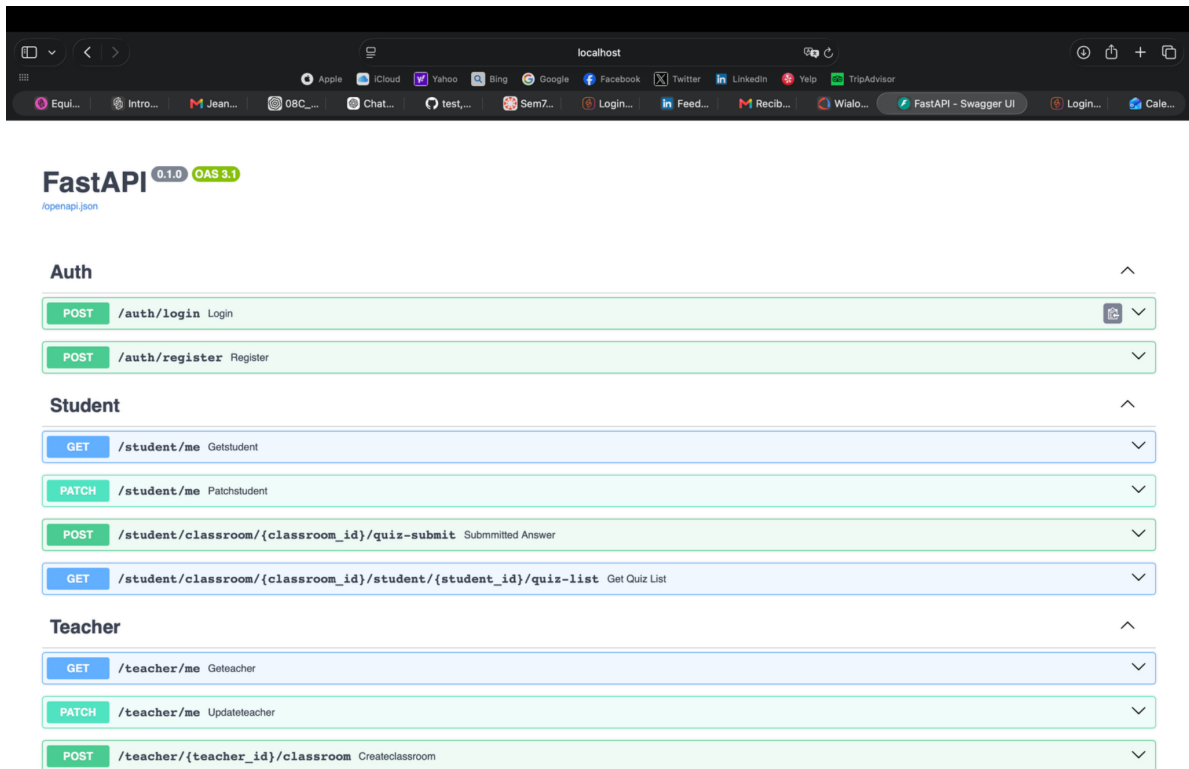


Figura 6: Swagger UI del orquestador disponible en localhost:8000/docs.

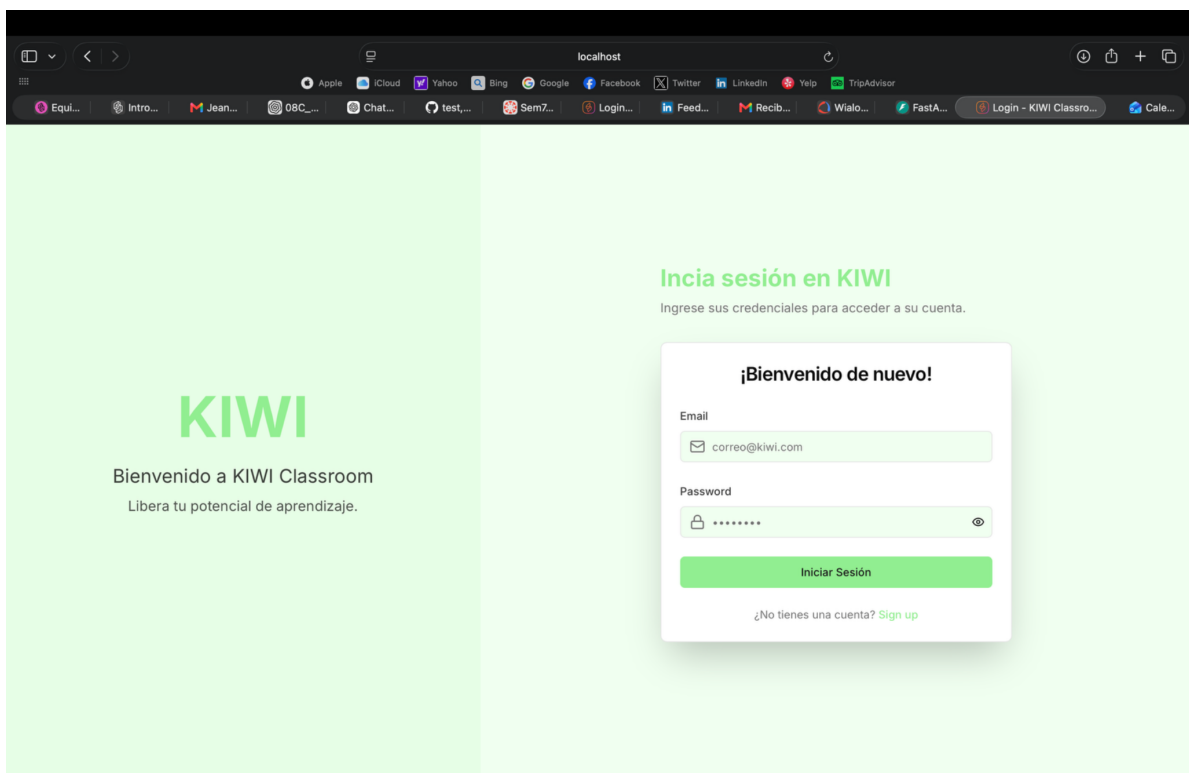


Figura 7: Frontend de KIWI disponible en localhost:5174/login.

8 Conclusiones

El uso de Conventional Commits y Pull Requests permite reconstruir el crecimiento del sistema y distinguir funcionalidades, correcciones, infraestructura, pruebas y documentación. La estrategia de ramas cortas mantiene a `main` como punto de integración y resulta coherente con Trunk Based Development.

Se implementaron 12 pruebas unitarias aisladas, un script Bash para automatizar operaciones repetitivas y un `compose.yml` que orquesta frontend, orquestador, microservicios y PostgreSQL con redes, persistencia, healthcheck y manejo de credenciales fuera del repositorio. Las pruebas unitarias finalizan satisfactoriamente y tanto el frontend como la documentación Swagger del orquestador fueron comprobados en ejecución local.